

Agent 后端接入交接文档

状态：Draft / Doc only

读者：C++ 后端开发者，以及第一次接触 C++ 与 TypeScript 进程协作的人。

本文件只说明接入边界、数据契约和后端需要实现的能力。当前任务禁止修改后端代码：不要改 `src/`、`include/`、数据库 schema、`message.proto`、主 `Makefile` 或现有 AI 回复链路。

0. 先理解：不是一种语言直接调用另一种语言

C++ 不能直接 `#include` 一个 TypeScript 文件，TypeScript 也不能直接调用 C++ 内存里的普通函数。

多语言结合通常靠这三层：

C++ 本地函数

- > 把 C++ struct 转成 JSON / protobuf / IPC message
- > 发给另一个进程
- > TypeScript 进程收到 message 后调用自己的 TS 函数

反过来也一样：

TypeScript 本地函数

- > 把参数转成 JSON / protobuf / IPC message
- > 发给 C++ 后端进程
- > C++ 后端收到 message 后调用自己的 C++ 函数

所以本文里说“后端调用 agent”，真实意思不是 C++ 直接执行 TS 函数，而是：

C++ AiReplyTask

- > `AgentClient::runTurn(req)`
- > HTTP / IPC / stdin-stdout
- > Node/TypeScript agent service
- > `AgentRuntime.runTurn(agent, turn)`
- > JSON result
- > C++ `AgentClient::runTurn(...)` 返回 result

本文里说“agent 调用后端函数”，真实意思也不是 TS 直接跳进 C++ 函数栈，而是：

TS ConversationContextStore.load(conversationId)

- > `BackendStoreClient.post("/internal/agent/context/load", ...)`
- > C++ internal handler
- > `load_agent_conversation_context(...)`

```
-> JSON result  
-> TS store adapter 返回 ConversationContextState
```

你可以把跨语言交互理解成“两个服务互相发结构化消息”。语言不重要，协议才重要。

1. 当前 agent 边界

Agent 现在是根目录 `agent/` 下的 TypeScript 子项目，不在后端 `src/ / include/` 里。

当前已经有：

- `agent/src/core/`：AgentProfile、TurnContext、AgentResponse 等核心类型。
- `agent/src/runtime/`：AgentRuntime.runTurn(agent, turn) 单轮执行入口。
- `agent/src/context/`：Atrium 双轴上下文。
- `agent/src/prompt/`：五段式 prompt 拼装。
- `agent/src/providers/`：ModelGateway 抽象。
- `agent/src/bridge/`：C++/JSON 到 TS core 的过渡契约。
- `agent/tests/`：纯逻辑 smoke tests。

当前还没有：

- 可由 C++ 调用的 agent 进程入口。
- C++ 后端的 agent repository 实现。
- C++ 后端的 AgentClient。
- MySQL 表。
- 主聊天链路接入。

后端现在不要把 `agent/src` 加进 C++ 编译链。TypeScript agent 应该作为独立进程或独立服务运行。

2. Agent 给后端什么，后端给 agent 什么

Agent 已经给出的东西

Agent 侧已经定义了后端接入的语言和边界：

```
agent/src/bridge/backendTransition.ts  
BACKEND_AGENT_PROTOCOL_VERSION = "atrium.agent.turn.v1"  
BackendAgentRunRequestPayload  
BackendAgentRunResultPayload  
normalizeBackendRunRequest(...)
```

它告诉后端：

- 一次 agent turn 需要哪些字段。
- 后端传来的字段名用 snake_case。
- TS core 内部字段名用 camelCase。
- 后端数字 ID 进入 TS core 前必须归一化为 string，避免 JavaScript 53-bit number 精度问题。

Agent 侧还定义了运行时需要的依赖：

```
AgentRuntime
  needs ModelGateway
  may use ConversationContextStore
  may use AgentStanceHistoryStore
  may use MemoryStore later
```

后端需要给 agent 的东西

后端不是给 agent “业务大脑”，而是给 agent 运行一轮所需的事实和能力：

1. 当前 turn 的基本信息：`room_id`、`conversation_id`、`trigger_message_id`、`context_until_message_id`、`user_id`。
2. 当前 AI 的身份：`ai_id`、`provider`、`model`、`display_name`、`thinking_adapter`。
3. 最近消息窗口：触发消息之前到 `context_until_message_id` 的可见消息。
4. 对话轴共享白板：按 `conversation_id` 读取 `ConversationContextState`。
5. 主体轴私有履历：按 `(conversation_id, ai_id)` 读取当前 AI 自己的 `AgentStanceHistory`。
6. `provider` 调用能力：后续用 `ModelGateway` 包装 DeepSeek / Qwen 等模型。
7. `commit` 能力：把 agent 的 `reply` / `no_reply` / `failed` 映射回现有 hidden message、落库、广播、错误帧语义。

Agent 返回给后端的東西

Agent 一轮最终返回 `AgentResponse`：

```
export interface AgentResponse {
  decision: "no_reply" | "reply" | "use_tool" | "needs_context" | "failed";
  content: string;
  error: string;
}
```

后端只按 `decision` 处理：

- `reply`：把 `content` 写入已有 hidden AI message，广播可见 AI 回复，继续现有接力。
- `no_reply`：不广播可见消息，不把 hidden message 变成空可见消息。
- `failed`：走现有 AI 错误帧 / 日志路径，不伪装成正常回复。
- `use_tool` / `needs_context`：当前不接主链路，先作为未来扩展保留。

3. 推荐的第一版进程关系

第一版建议用 HTTP loopback，因为最容易调试：

```
C++ WebServer process
  listens normal public HTTP / WebSocket
  owns MySQL / Redis / room membership / scheduler / hidden message /
  broadcast
```

```
Node TypeScript agent process
  listens only 127.0.0.1:<agent_port>
  owns AgentRuntime / prompt / context assembly / model gateway adapter
```

后端调用 agent:

```
POST http://127.0.0.1:<agent_port>/run-turn
```

agent 如需调用后端内部函数，可以二选一：

1. 简单模式：C++ 在 `/run-turn` 请求里预先放好 messages、conversation context、stance history。TS agent 本轮不反向调用后端。
2. Adapter 模式：TS 的 `ConversationContextStore` / `AgentStanceHistoryStore` 通过 loopback internal API 反向请求 C++ 后端。

对第一次接触多语言交互的人，建议先做简单模式。它只有一个方向：

```
C++ -> TS -> C++
```

等你确认 JSON 契约、ID 转换、reply/no_reply 语义都通了，再做 Adapter 模式：

```
C++ -> TS
      TS -> C++ load context
      TS -> C++ append stance
<- TS result
```

4. 后端调用 agent 的接入点

未来接入点不是新 HTTP API，也不是前端 WebSocket 协议，而是当前 AI 回复任务链路。

目标位置：

```
ConvAiScheduler::submit()
-> insert_hidden_message()
-> AiReplyTask 入队
-> AiReplyTask::process()
-> 当前调用 AiClient/provider
-> 未来这里包一层 AgentClient::runTurn()
```

`SubReactor` / `AiReplyTask` 只应该负责编排：

1. 已有调度。
2. 已有 hidden message。

3. 构造 `AgentRunRequest`。
4. 调 `AgentClient::runTurn()`。
5. 根据 `AgentRunResult` 走现有落库/广播/静默/错误路径。

不要把 SQL 细节、JSON 传输、prompt 拼装塞进 `SubReactor`。

5. Backend -> Agent 请求契约

当前 TS bridge 已有基础 payload:

```
export const BACKEND_AGENT_PROTOCOL_VERSION = "atrium.agent.turn.v1";

export interface BackendAgentRunRequestPayload {
  protocol: typeof BACKEND_AGENT_PROTOCOL_VERSION;
  ref: BackendAgentTurnRefPayload;
  agent: BackendAgentProfilePayload;
  turn: BackendTurnContextPayload;
}
```

后端第一版请求 JSON 建议长这样。ID 建议用字符串发给 TS, 即使它们在 C++ 里是 `uint64_t`:

```
{
  "protocol": "atrium.agent.turn.v1",
  "ref": {
    "room_id": "1",
    "conversation_id": "42",
    "trigger_message_id": "1001",
    "context_until_message_id": "1001",
    "user_id": "7"
  },
  "agent": {
    "id": "12",
    "provider": "deepseek",
    "model": "deepseek-v4-flash",
    "display_name": "DeepSeek",
    "thinking_adapter": "counterexample",
    "custom_thinking_instruction": ""
  },
  "turn": {
    "room_id": "1",
    "conversation_id": "42",
    "trigger_message_id": "1001",
    "context_until_message_id": "1001",
    "user_id": "7",
    "source": "user_message",
    "messages": [
      {
        "id": "998",
        "sender": {
          "id": "7",
```

```
        "kind": "user",
        "display_name": "lyc"
      },
      "content": "我们继续讨论 agent 上下文工程。"
    ]
  }
}
```

字段说明：

字段	后端来源	用途
ref.room_id	当前房间	commit / 日志 / 追踪
ref.conversation_id	当前 conversation	读取对话轴和主体轴
ref.trigger_message_id	触发本轮 AI 的消息	prompt 中最后的当前消息
ref.context_until_message_id	本轮允许看到的最后消息	防止同轮 AI 互相锚定
ref.user_id	触发用户	权限、审计、owner 操作扩展
agent.id	AI 成员 ID	读取当前 AI 自己的主体轴
agent.provider / agent.model	AI 配置	provider adapter
agent.thinking_adapter	AI 思维取向	prompt 静态层
turn.messages	后端最近消息窗口	prompt 原文窗口

context_until_message_id 很重要。多 AI 同一轮独立思考时，AI B 不应该看到 AI A 在同一触发消息之后刚刚生成的内容。

6. Agent -> Backend 返回契约

当前 TS bridge 的结果类型：

```
export interface BackendAgentRunResultPayload {
  protocol: typeof BACKEND_AGENT_PROTOCOL_VERSION;
  ref: AtriumTurnRef;
  agent: AgentProfile;
  response: AgentResponse;
}
```

返回 JSON 示例：

```
{
  "protocol": "atrium.agent.turn.v1",
  "ref": {
    "roomId": "1",
```

```

    "conversationId": "42",
    "triggerMessageId": "1001",
    "contextUntilMessageId": "1001",
    "userId": "7"
  },
  "agent": {
    "id": "12",
    "provider": "deepseek",
    "model": "deepseek-v4-flash",
    "displayName": "DeepSeek",
    "thinkingAdapter": "counterexample"
  },
  "response": {
    "decision": "reply",
    "content": "我会先把问题拆成两层：进程协议和业务上下文。",
    "error": ""
  }
}

```

C++ 处理伪代码：

```

if (result.response.decision == AgentDecision::NoReply) {
  // 等价于当前 AiClientStatus::NoReply :
  // 不广播可见消息，不把 hidden message complete 成空消息。
  return;
}

if (result.response.decision == AgentDecision::Reply) {
  complete_message_content(ai_message_id, result.response.content);
  broadcast_ai_reply(ai_message_id);
  dispatch_to_other_ais_if_needed();
  return;
}

if (result.response.decision == AgentDecision::Failed) {
  emit_ai_stream_error(...);
  return;
}

```

注意：主体轴履历 `append_agent_stance_history(...)` 只能写一次。

- 如果 TS runtime 配了 `AgentStanceHistoryStore` 后端 adapter，则 agent 在 `runTurn()` 内部已经调用 `append`，C++ commit 阶段不要再追加。
- 如果第一版没有 TS store adapter，后端可以在 `reply commit` 后追加一次作为过渡方案。
- 如果要记录准确的 `response_message_id`，接入前需要让 agent 请求里携带 hidden AI message id，或者让后端在 `commit` 后补齐。当前 TS runtime 的 `append` 只保证有 `triggerMessageId` 和内容。

7. 后端需要实现的本地 C++ 函数

这些函数不是给 TypeScript 直接链接调用的。它们是 C++ 后端自己的 repository / adapter 函数，之后由 HTTP/IPC handler 或 `AgentClient` 包起来。

建议新增：

```
include/agent_context_repository.h
src/agent_context_repository.cpp
```

只做数据库读写，不处理 WebSocket，不拼 prompt。

```
enum class AgentConversationPhase : uint8_t {
    Divergence = 0,
    ConvergenceExecution = 1,
    Blocked = 2,
};

enum class AgentContextEntryKind : uint8_t {
    Goal = 0,
    Constraint = 1,
    Decision = 2,
    RejectedOption = 3,
    OpenQuestion = 4,
    Risk = 5,
    KeyFact = 6,
    ProgressNote = 7,
};

enum class AgentContextEntryStatus : uint8_t {
    Active = 0,
    Superseded = 1,
    Resolved = 2,
    Rejected = 3,
};

struct AgentContextSource {
    uint64_t message_id = 0;
    std::string note;
};

struct AgentRejectedOption {
    std::string option;
    std::string reason;
    std::string premise;
};

struct AgentContextEntry {
    uint64_t id = 0;
    uint64_t conversation_id = 0;
    AgentContextEntryKind kind = AgentContextEntryKind::KeyFact;
    AgentContextEntryStatus status = AgentContextEntryStatus::Active;
```



```

    std::string content;
    std::optional<AgentRejectedOption> rejected_option;
    uint32_t priority = 0;
    uint64_t created_at_ms = 0;
    uint64_t updated_at_ms = 0;
    std::vector<AgentContextSource> sources;
};

struct AgentConversationContext {
    uint64_t conversation_id = 0;
    AgentConversationPhase phase = AgentConversationPhase::Divergence;
    std::string summary;
    uint64_t last_summarized_message_id = 0;
    uint64_t updated_at_ms = 0;
    std::vector<AgentContextEntry> entries;
};

struct AgentStanceRecord {
    uint64_t id = 0;
    uint64_t conversation_id = 0;
    uint64_t ai_id = 0;
    uint64_t trigger_message_id = 0;
    uint64_t response_message_id = 0;
    std::string content;
    uint64_t created_at_ms = 0;
};

```

Repository 函数:

```

uint64_t now_ms();

MysqlPool::QueryResult load_agent_conversation_context(
    uint64_t conversation_id,
    AgentConversationContext& out
);

MysqlPool::QueryResult save_agent_conversation_context(
    const AgentConversationContext& context
);

MysqlPool::QueryResult set_agent_conversation_phase(
    uint64_t conversation_id,
    AgentConversationPhase phase,
    uint64_t owner_user_id,
    uint64_t updated_at_ms
);

MysqlPool::QueryResult insert_agent_rejected_option(
    uint64_t conversation_id,
    const std::string& option,
    const std::string& reason,
    const std::string& premise,

```

```

        uint64_t source_message_id,
        uint64_t owner_user_id,
        uint64_t created_at_ms
    );

MysqlPool::QueryResult load_agent_stance_history(
    uint64_t conversation_id,
    uint64_t ai_id,
    size_t limit,
    std::vector<AgentStanceRecord>& out
);

MysqlPool::QueryResult append_agent_stance_history(
    uint64_t conversation_id,
    uint64_t ai_id,
    uint64_t trigger_message_id,
    uint64_t response_message_id,
    const std::string& content,
    uint64_t created_at_ms
);

```

后端权限要求：

- `set_agent_conversation_phase()` 只能 owner 触发。
- `insert_agent_rejected_option()` 只能 owner 触发，并且 `option/reason/premise` 必须同时写。
- `load_agent_stance_history()` 必须同时带 `conversation_id + ai_id`，禁止只按 `conversation_id` 查。
- AI A 的主体轴不能传给 AI B。

8. Agent 如何调用后端实现的函数

这里用 Adapter 模式解释。即使第一版先做简单模式，也应该理解这层。

TS runtime 只认识接口：

```

export interface ConversationContextStore {
    load(conversationId: ConversationId): ConversationContextState |
    undefined | Promise<ConversationContextState | undefined>;
    save(state: ConversationContextState): ConversationContextWriteResult |
    Promise<ConversationContextWriteResult>;
}

export interface AgentStanceHistoryStore {
    load(conversationId: ConversationId, agentId: AgentId):
    AgentStanceHistory | undefined | Promise<AgentStanceHistory | undefined>;
    append(record: AppendAgentStanceHistoryRecord):
    ConversationContextWriteResult | Promise<ConversationContextWriteResult>;
}

```

要让 TS agent 使用 C++ 后端函数，不是让 TS 直接链接 C++，而是写一个 TS adapter：

```
class BackendConversationContextStore implements ConversationContextStore {
  async load(conversationId: string) {
    return postJson("/internal/agent/context/load", { conversation_id:
conversationId });
  }

  async save(state: ConversationContextState) {
    return postJson("/internal/agent/context/save",
toBackendContextPayload(state));
  }
}

class BackendStanceHistoryStore implements AgentStanceHistoryStore {
  async load(conversationId: string, agentId: string) {
    return postJson("/internal/agent/stance/load", {
      conversation_id: conversationId,
      ai_id: agentId,
      limit: 6
    });
  }

  async append(record: AppendAgentStanceHistoryRecord) {
    return postJson("/internal/agent/stance/append",
toBackendStancePayload(record));
  }
}
```

C++ 后端则实现对应 internal handler：

```
POST /internal/agent/context/load
-> parse JSON
-> load_agent_conversation_context(conversation_id, out)
-> return JSON

POST /internal/agent/stance/load
-> parse JSON
-> load_agent_stance_history(conversation_id, ai_id, limit, out)
-> return JSON

POST /internal/agent/stance/append
-> parse JSON
-> append_agent_stance_history(...)
-> return JSON
```

这样 TS 看到的是普通 TypeScript interface，C++ 看到的是普通 C++ repository 函数，中间只靠 JSON 协议连接。

9. 后端怎么写 AgentClient

建议新增：

```
include/agent_client.h
src/agent_client.cpp
```

只负责和 agent 进程通信，不访问 MySQL，不拼 prompt。

职责：

1. 把 C++ `AgentRunRequest` 转成 `BackendAgentRunRequestPayload` JSON。
2. 通过 HTTP loopback 或 IPC 发给 TypeScript agent。
3. 校验 `protocol == "atrium.agent.turn.v1"`。
4. 把 response JSON 转回 C++ `AgentRunResult`。
5. 对网络错误、超时、协议错误返回明确状态。

伪接口：

```
enum class AgentDecision {
    NoReply,
    Reply,
    UseTool,
    NeedsContext,
    Failed,
};

struct AgentRunResult {
    AgentDecision decision = AgentDecision::Failed;
    std::string content;
    std::string error;
};

class AgentClient {
public:
    AgentClient(std::string endpoint, uint32_t timeout_ms);

    MysqlPool::QueryResult runTurn(
        const AgentRunRequest& request,
        AgentRunResult& out
    );
};
```

如果用 HTTP，仓库已有 `third_party/httpplib.h` 和 `third_party/json.hpp`，可以复用现有风格。

10. 数据库表需求

当前主线只做 Atrium 双轴 context，不做泛化长期 memory。

需要四张表：

- `conversation_context_states`：每个 conversation 一行，保存 summary、phase、摘要进度。
- `conversation_context_entries`：目标、约束、决策、否定方案、开放问题、风险、关键事实、进展。
- `conversation_context_entry_sources`：条目来源消息锚点。
- `agent_stance_history_records`：每个 AI 在每场 conversation 内自己的私有履历。

建议 schema：

```
CREATE TABLE conversation_context_states (
  conversation_id BIGINT UNSIGNED NOT NULL COMMENT '应用层外键 ->
  conversations.id',
  phase TINYINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '0=发散期, 1=收敛执行期,
  2=撞墙期',
  summary TEXT NOT NULL,
  last_summarized_message_id BIGINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '摘
  要覆盖到的最后消息 id',
  updated_at_ms BIGINT UNSIGNED NOT NULL,

  PRIMARY KEY (conversation_id),
  KEY idx_context_updated (updated_at_ms)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4
COMMENT = '单个 conversation 的长期上下文摘要状态';

CREATE TABLE conversation_context_entries (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  conversation_id BIGINT UNSIGNED NOT NULL COMMENT '应用层外键 ->
  conversations.id',
  kind TINYINT UNSIGNED NOT NULL COMMENT '0=GOAL, 1=CONSTRAINT, 2=DECISION,
  3=REJECTED_OPTION, 4=OPEN_QUESTION, 5=RISK, 6=KEY_FACT, 7=PROGRESS_NOTE',
  status TINYINT UNSIGNED NOT NULL COMMENT '0=ACTIVE, 1=SUPERSEDED,
  2=RESOLVED, 3=REJECTED',
  content TEXT NOT NULL,
  rejected_option TEXT DEFAULT NULL COMMENT 'kind=REJECTED_OPTION 时必填',
  rejection_reason TEXT DEFAULT NULL COMMENT 'kind=REJECTED_OPTION 时必填',
  rejection_premise TEXT DEFAULT NULL COMMENT 'kind=REJECTED_OPTION 时必填',
  priority INT UNSIGNED NOT NULL DEFAULT 0,
  created_at_ms BIGINT UNSIGNED NOT NULL,
  updated_at_ms BIGINT UNSIGNED NOT NULL,
  deleted_at_ms BIGINT UNSIGNED DEFAULT NULL,

  PRIMARY KEY (id),
  KEY idx_context_entries_active (conversation_id, status, kind,
  deleted_at_ms, priority, updated_at_ms),
  KEY idx_context_entries_kind (kind, status)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4
COMMENT = '单个 conversation 的结构化上下文条目';

CREATE TABLE conversation_context_entry_sources (
  entry_id BIGINT UNSIGNED NOT NULL COMMENT '应用层外键 ->
  conversation_context_entries.id',
```

```

message_id BIGINT UNSIGNED NOT NULL COMMENT '应用层外键 -> messages.id',
note VARCHAR(255) DEFAULT NULL,

PRIMARY KEY (entry_id, message_id),
KEY idx_context_source_message (message_id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4
COMMENT = 'conversation context 条目的来源消息锚点';

CREATE TABLE agent_stance_history_records (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  conversation_id BIGINT UNSIGNED NOT NULL COMMENT '应用层外键 ->
conversations.id',
  ai_id BIGINT UNSIGNED NOT NULL COMMENT '应用层外键 -> ai.id /
conversation_ai_members.ai_id',
  trigger_message_id BIGINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '触发本轮 AI
发言的消息',
  response_message_id BIGINT UNSIGNED NOT NULL DEFAULT 0 COMMENT 'AI 回复落
库后的消息 id',
  content MEDIUMTEXT NOT NULL COMMENT '第一版存该 AI 本轮发言原文或简单摘要',
  created_at_ms BIGINT UNSIGNED NOT NULL,

PRIMARY KEY (id),
KEY idx_stance_history_agent_conversation (conversation_id, ai_id, id),
KEY idx_stance_history_trigger (trigger_message_id),
KEY idx_stance_history_response (response_message_id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4
COMMENT = '每个 AI 在单场 conversation 内的私有立场履历';

```

第一版 `save_agent_conversation_context()` 可以整体替换 `entries`:

1. upsert `conversation_context_states`。
2. soft delete 该 conversation 原有 `entries`。
3. insert 新 `entries`。
4. insert 每个 entry 的 `sources`。

后续如果担心并发覆盖，再做 patch 级别更新。

11. 一个完整例子

场景：用户在 conversation 42 发了消息 1001，调度器决定让 AI 12 回复。

C++ 后端准备

```

room_id = 1
conversation_id = 42
trigger_message_id = 1001
context_until_message_id = 1001
user_id = 7
ai_id = 12
hidden_ai_message_id = 1002

```

后端做三件事：

```
load_recent_messages(conversation_id, context_until_message_id, 30,
req.messages);
load_agent_conversation_context(conversation_id, req.conversation_context);
load_agent_stance_history(conversation_id, ai_id, 6, req.stance_history);
```

然后调用：

```
AgentRunResult result;
AgentClient client("http://127.0.0.1:18090/run-turn", 30000);
auto ret = client.runTurn(req, result);
```

TS agent 内部发生什么

```
normalizeBackendRunRequest(payload)
-> AgentRuntime.runTurn(agent, turn)
-> buildContextPack(turn)
-> appendStaticAgentIdentityToPrompt(...)
-> ConversationContextStore.load(conversation_id)
-> appendConversationContextToPrompt(...)
-> append recent messages
-> AgentStanceHistoryStore.load(conversation_id, ai_id)
-> appendPrivateStanceToPrompt(...)
-> append current trigger message
-> ModelGateway.complete(...)
-> AgentResponse
```

C++ 后端提交结果

如果返回：

```
{ "decision": "reply", "content": "我建议先固定协议，再接数据库。", "error": ""
}
```

后端：

```
complete_message_content(hidden_ai_message_id, result.content);
broadcast_ai_reply(hidden_ai_message_id);
dispatch_to_other_ais_if_needed();
```

如果返回：

```
{ "decision": "no_reply", "content": "", "error": "" }
```

后端：

```
// 不广播，不生成空消息。  
return;
```

12. 接入前 agent 侧还要补的东西

后端真正开始写接入代码前，agent 侧还需要定稿这些点：

1. 进程入口：`agent` 是 HTTP loopback 服务，还是长期子进程 IPC。
2. `AgentClient` 的传输协议：HTTP path、超时、错误码、日志字段。
3. `ModelGateway` 第一版怎么接 DeepSeek/Qwen：TS 直接调 provider，还是先回调 C++ provider adapter。
4. `ConversationContextStore` / `AgentStanceHistoryStore` 是简单模式预加载，还是 Adapter 模式反向调用后端。
5. 主体轴 append 由谁写：TS runtime store adapter 写，还是 C++ commit 后写。只能选一个，不能重复。
6. 是否把 `hidden_ai_message_id` / `response_message_id` 加进 bridge payload，方便主体轴记录准确回复消息 ID。

这些是 agent 侧接入任务，不要求后端开发者凭空补齐业务判断。

13. 当前不要做

- 不要修改主后端代码来“试接一下”。
- 不要把 TypeScript 文件加入 C++ `Makefile`。
- 不要让 `SubReactor` 直接拼 prompt。
- 不要让 runtime 直接认识 MySQL、Redis、WebSocket、Reactor。
- 不要把 AI A 的主体轴历史传给 AI B。
- 不要让 AI 自动写 owner-only 的决策区、被否方案或阶段切换。
- 不要把泛化长期 `MemoryStore` 当成当前 Atrium 双轴 context 主线。
- 不要让 `<NO_REPLY>` 产生可见空消息。

14. 给第一次做这个的人：你只需要先掌握一件事

第一阶段不要试图“一下子懂完整业务”。先把语言交互这件事想清楚：

```
C++ 想用 TS agent  
= C++ 写一个本地 AgentClient  
= AgentClient 发 JSON 给 Node/TS 进程  
= TS 进程返回 JSON  
= C++ 把 JSON 结果放回现有后端链路
```


等这一层通了，再理解双轴 context：

```
conversation_id 一份共享白板  
(conversation_id, ai_id) 一份当前 AI 私有履历
```

这就是 Atrium 和传统单 agent 上下文工程最大的差别：不是一个 AI 记住一段聊天，而是多人多 AI 的讨论空间里，公共事实和每个 AI 自己的思考轨迹必须分开。